

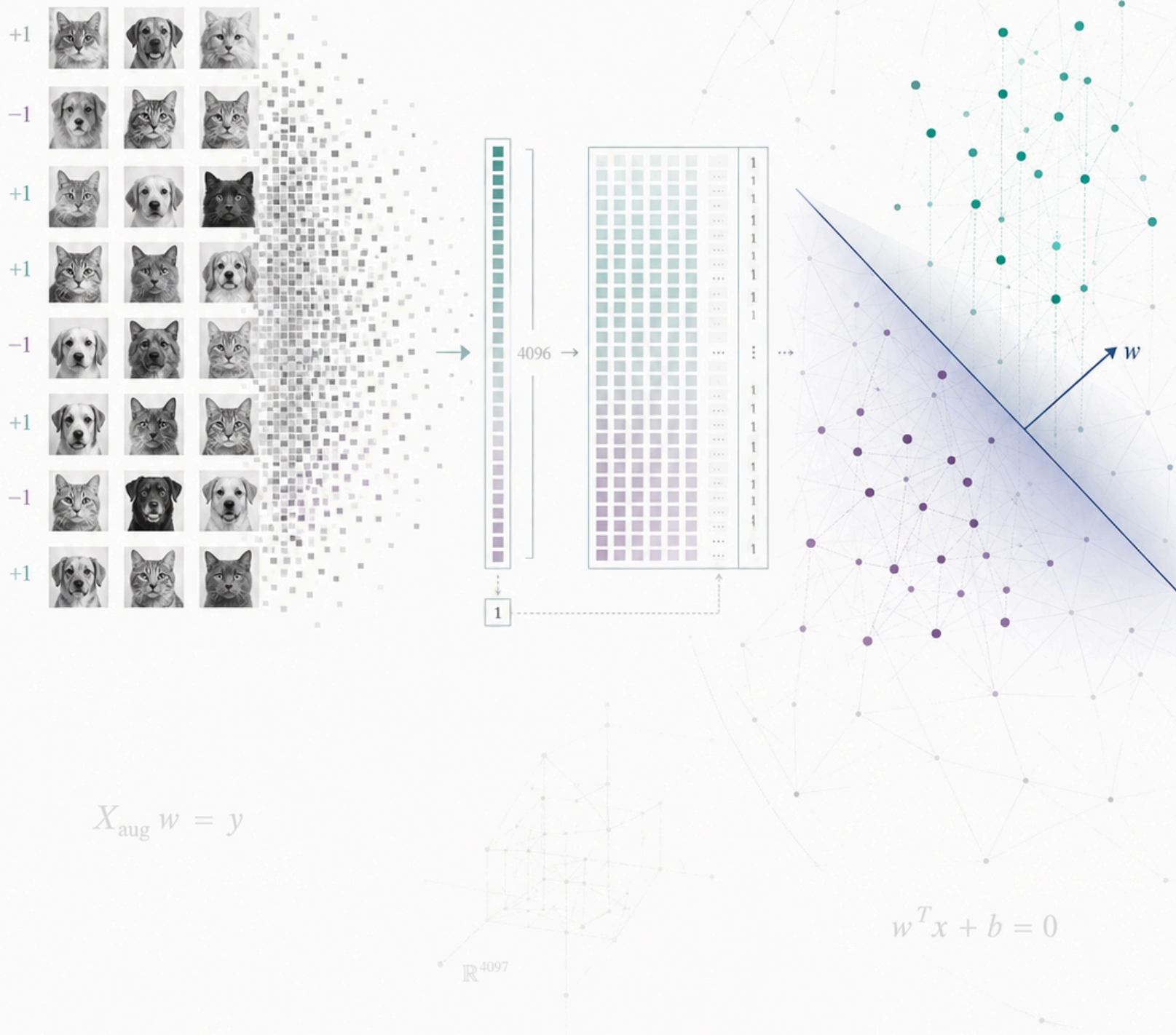
High-Dimensional Perceptron Memorization

$64 \times 64 = 4096$

$d + 1 = 4097$

Why 4096 pixels plus a bias can fit random labels

Hunter Colson



1 Why this chapter exists

Imagine taking a folder of cat and dog face photographs, shrinking every image to a 64 by 64 grayscale square, and handing those numbers to one of the simplest classifiers in machine learning: a single perceptron. If the labels still mean cat and dog, good training accuracy is tempting to read as evidence that the model has found something visual. Perhaps it has noticed ears, eyes, snouts, fur texture, or some other pattern we would recognize.

This chapter deliberately removes that comfort. After the images are processed, the true labels are thrown away and replaced with random $+1$ and -1 targets. Some cats are assigned $+1$, some cats are assigned -1 , and the same thing happens to the dogs. The photographs remain real, but the target attached to each photograph no longer means cat or dog. It is just a coin flip written beside an image.

For the processed images used here, a perceptron can still fit those random labels perfectly through 4097 training examples. That result says nothing about recognizing cats, because the labels no longer contain a cat concept to recognize. It says the model has enough geometric freedom to assign the requested signs to the particular training images in front of it. Perfect training accuracy, by itself, is therefore a weaker achievement than it first appears: sometimes it reflects a useful rule, and sometimes the input space simply has enough room for memorization.

The point is not that perceptrons are bad, or that high-dimensional models are useless, or that VC dimension explains all of modern deep learning. The point is narrower and more useful: when the number of examples is small relative to the number of input dimensions, even a simple linear classifier can have surprising capacity to memorize. Random labels make that capacity visible because they remove the real-world pattern we would otherwise be tempted to credit.

2 The experiment

The experiment uses the AFHQ animal-faces dataset and keeps the mechanics intentionally simple. The code loads cat and dog images, resizes them to 64 by 64 grayscale, flattens each image into a row of 4096 numbers, assigns random binary labels, and then asks the same question in two complementary ways. One route is algebraic: can we directly construct weights that classify every randomized training image correctly? The other route is algorithmic: can the perceptron learning rule find such weights by making one correction after another?

The algebraic route is the exact separator construction. It builds a matrix from the images, adds one bias column, checks whether the rows of that matrix have the right rank, and then asks for weights whose scores match the target labels exactly:

$$X_{\text{aug}}\tilde{w} = y. \quad (1)$$

The equation asks whether there is one parameter vector, $\tilde{w}$, whose scores exactly match the label vector y . If the score for each image is exactly $+1$ or -1 , then the sign of every score is automatically correct, and the perceptron has zero training error on the randomized labels.

The training route uses the perceptron learning rule instead of solving the equation directly. It starts with weights at zero, looks at one image at a time, and updates the weights only when the model makes a mistake. Once the algebra tells us a separator exists, the training run asks a more practical question: how hard is it for the original mistake-driven algorithm to find one? Figure 1 shows how the images move through both tests.

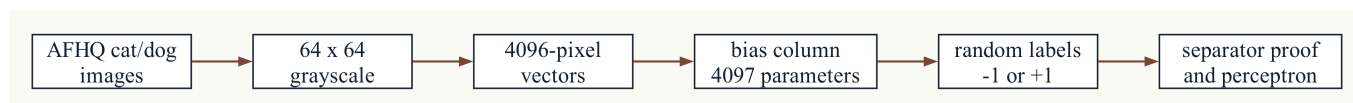


Figure 1: The experiment turns images into a high-dimensional linear-algebra problem. Real cat and dog images are resized, flattened into 4096-pixel vectors, augmented with a bias column, assigned random labels, and then passed to both an exact separator proof and perceptron training.

3 How to read the experiment

This chapter assumes no machine learning background. The math is included because the claim is mathematical, but each symbol is introduced where it becomes useful. The idea builds in layers: an image becomes a vector, a perceptron becomes a weighted sum over that vector, and the bias term becomes one more column in a matrix. Once those pieces are in place, the VC-dimension statement stops sounding like a slogan and becomes a concrete claim about how many independent constraints a linear model can satisfy.

By the end, the symbols should no longer feel like decoration. You should know what x is, what w is, why b can be treated as another weight, what rank is checking, why $X_{\text{aug}}\tilde{w} = y$ proves that a separator exists, why the perceptron learning rule can still take many epochs, and why training accuracy alone cannot tell us whether the model learned a meaningful concept. The thread is simple once the notation is unpacked: a perceptron is a weighted sum, an image is a long list of pixels, and a high-dimensional list gives the weighted sum many adjustable directions. If the examples are independent enough, those directions can be used to fit arbitrary labels.

4 The capacity claim

A 64 by 64 grayscale image contains

$$64 \times 64 = 4096$$

pixels. Flattening does not change those values; it only changes their arrangement. The square image becomes one long row of numbers, which we write as

$$x = (x_1, x_2, \dots, x_{4096}).$$

The symbol x means one input image after it has been converted into numbers. The value x_1 is the first pixel, x_2 is the second pixel, and the sequence continues until the last pixel, x_{4096} . The perceptron has one weight for each of those pixel positions, plus one bias term that shifts the decision boundary. Its raw score is

$$s = w_1x_1 + w_2x_2 + \dots + w_{4096}x_{4096} + b. \quad (2)$$

The values $w_1, \dots, w_{4096}$ are the pixel weights, b is the bias, and s is the signed score produced by the model. A positive score becomes the prediction $+1$; a negative score becomes the prediction -1 . The whole classifier is therefore controlled by 4097 adjustable numbers: 4096 pixel weights and one bias.

For linear classifiers in d input dimensions, the VC dimension is $d + 1$. Here $d = 4096$, so the boundary is $d + 1 = 4097$. Under the right rank condition, a linear classifier can fit any binary labels on up to 4097 examples, even labels assigned by random coin flips. That is why this experiment is useful: if the model fits the randomized labels, the success cannot be explained by cat features or dog features. The labels no longer point to animals. They point to capacity.

5 The data: animal faces become rows of numbers

The dataset is Kaggle's [andrewmvd/animal-faces](#), commonly known as AFHQ, short for Animal Faces-HQ. It contains high-resolution animal face images in cat, dog, and wildlife categories. This experiment uses only cats and dogs. The true animal class is used only to select the images and to make the figures readable; after that, the training labels are replaced with random -1 and $+1$ targets.

The original inputs are photographs, but the perceptron never receives them as photographs. Before training, each image goes through the same processing pipeline:

1. load the image;
2. convert it to grayscale;
3. resize it to 64 by 64 pixels;
4. scale the pixel values numerically;
5. flatten the image into a 4096-number row.

At first this can feel as if the image has disappeared. It has not. A 64 by 64 grid and a 4096-number row contain the same brightness values arranged in different shapes. The grid is easier for us to inspect

visually; the row is easier for the model to multiply by weights. For a tiny 3 by 3 grayscale image, the same rearrangement would look like this:

$$\begin{bmatrix} 12 & 40 & 91 \\ 8 & 55 & 110 \\ 4 & 61 & 130 \end{bmatrix} \longrightarrow (12, 40, 91, 8, 55, 110, 4, 61, 130).$$

A vector is just an ordered list of numbers. The order matters because each position corresponds to a location in the original image. Pixel 1 might be the upper-left corner. Pixel 4096 might be the lower-right corner.

Once every image has been turned into a vector, the training set becomes a table. Each row is one image. Each column is one pixel position. If there are N images, the table has N rows and 4096 pixel columns. We call that table X .

6 The perceptron as a small machine

The perceptron is a linear classifier. It takes the input numbers, multiplies each input by a corresponding weight, adds everything together, adds a bias, and then checks the sign.

[Figure 2](#) draws that computation as a small machine: pixel values enter on the left, weights scale them, the bias shifts the sum, and the sign of the final score becomes the prediction.

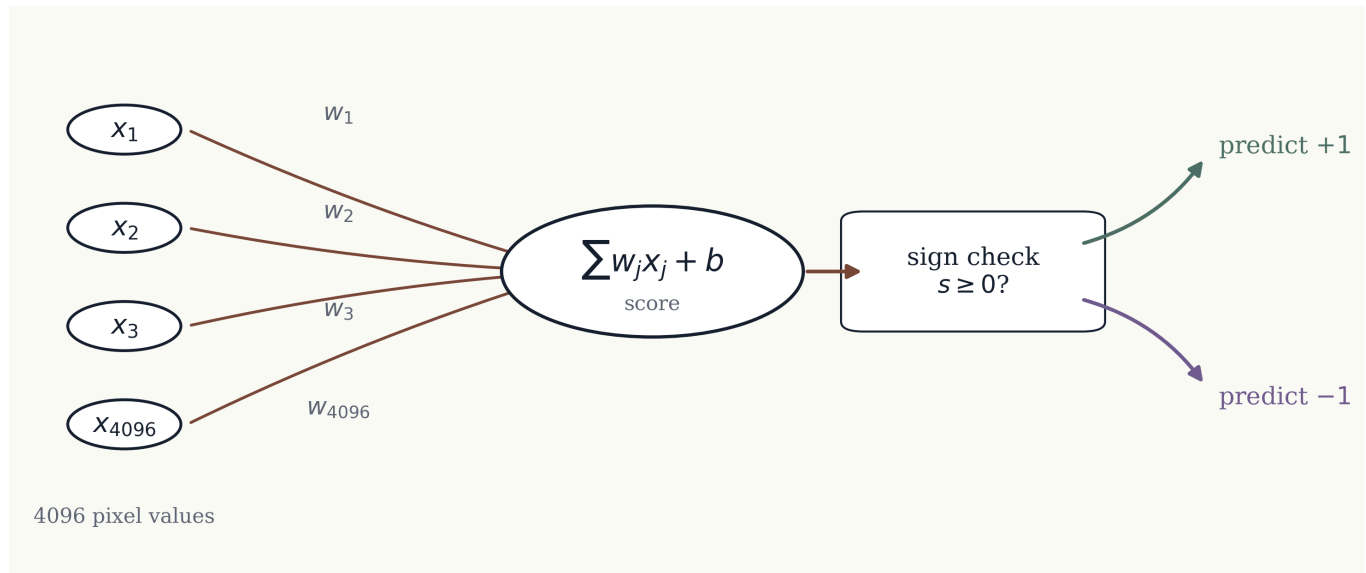


Figure 2: A perceptron multiplies each input feature by a weight, adds the weighted values plus the bias, and predicts by checking whether the score is positive or negative.

The word “linear” matters. The perceptron does not multiply pixels by other pixels. It does not build an eye detector, an ear detector, or a fur detector. It only computes the weighted sum in [Equation 2](#) and

then checks the sign.

In symbols, the prediction is

$$\hat{y} = \begin{cases} +1 & \text{if } w^\top x + b \geq 0, \\ -1 & \text{if } w^\top x + b < 0. \end{cases}$$

The symbol $\hat{y}$, pronounced “y-hat,” means the model’s predicted label. The true target label is written y , and in this experiment it is always either $+1$ or -1 . The entire learned model is contained in w and b : the vector w holds the pixel weights, and the scalar b holds the bias. Before training, those numbers are just starting values. During training, they are adjusted until the signs of the scores match the labels on the training set.

7 A line before a hyperplane

The 4096-dimensional case is impossible to draw honestly, so it helps to begin with a toy world that has only two inputs. In that smaller world, the perceptron score has the same structure as before, just with fewer terms:

$$s = w_1x_1 + w_2x_2 + b.$$

The decision boundary is the set of points where that score equals zero:

$$w_1x_1 + w_2x_2 + b = 0.$$

In two dimensions, that equation describes a line. Points on one side have positive scores, points on the other side have negative scores, and points exactly on the line are the cases where the model is undecided. Now take three points:

point	x_1	x_2	label
A	0	0	-1
B	1	0	+1
C	0	1	+1

One separator is:

$$2x_1 + 2x_2 - 1 = 0.$$

This means $w_1 = 2$, $w_2 = 2$, and $b = -1$. Check the score at each point:

$$\begin{aligned} A : 2(0) + 2(0) - 1 &= -1, \\ B : 2(1) + 2(0) - 1 &= +1, \\ C : 2(0) + 2(1) - 1 &= +1. \end{aligned}$$

The scores exactly equal the labels, so their signs are correct as well. The perceptron does not need the score to equal the label in order to classify correctly, but equality gives us a stronger and cleaner construction.

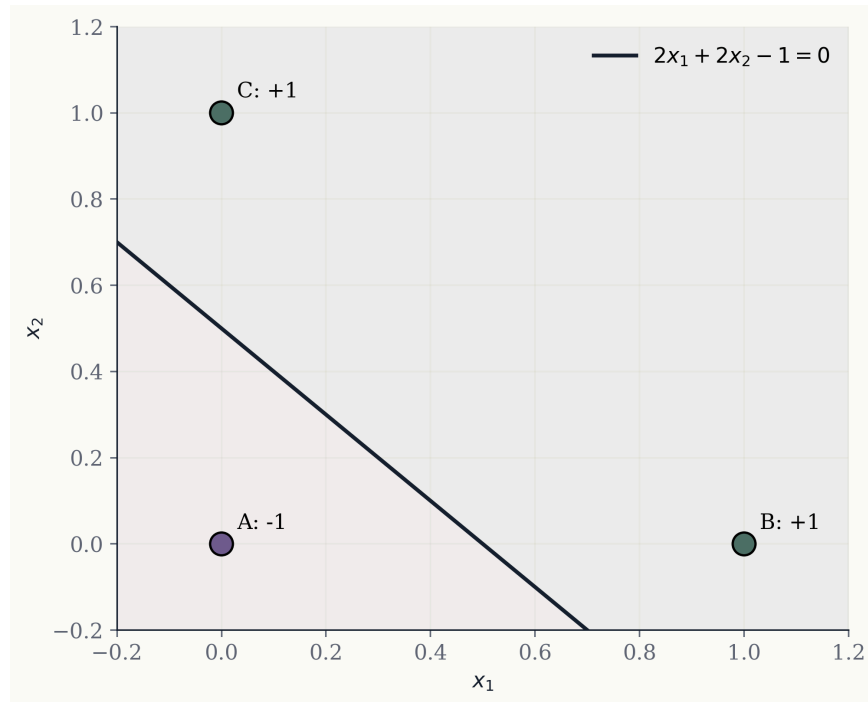


Figure 3: A line separating three points in two dimensions. The same logic scales to a hyperplane in 4096 dimensions.

This tiny example already contains the large result. There are two input dimensions plus one bias term, giving three adjustable numbers, and with three points in a favorable position those numbers can be chosen so the scores match arbitrary binary labels. For images, replace the two input dimensions with 4096. The same kind of separator becomes a hyperplane, which is the high-dimensional version of a line or plane.

8 What a perceptron cannot do

The perceptron is limited. A single perceptron can only draw one linear boundary. The classic failure case is XOR, where four points sit at the corners of a square with matching labels on opposite corners:

x_1	x_2	label
0	0	-1
1	0	+1
0	1	+1
1	1	-1

No single line can separate those labels, because whichever way the line is drawn, one corner from each class ends up on the wrong side.

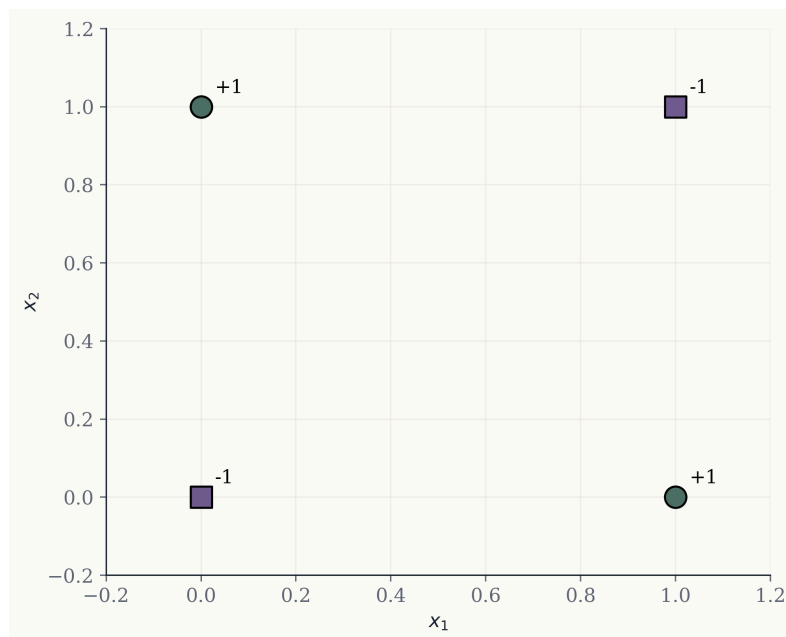


Figure 4: The XOR pattern is not linearly separable because matching labels sit on opposite corners of the square.

This is why the result in this chapter should be stated carefully. We are not saying a perceptron can learn every pattern. It cannot. We are saying that when the input dimension is large and the number of examples is not too large, linear separators can fit many arbitrary labelings.

That contrast is the reason XOR belongs in this chapter. XOR shows the limitation of a single linear boundary, while VC dimension shows how much freedom that same boundary can still have when the examples live in a high-dimensional space.

9 Random labels are the diagnostic

If we trained on true cat and dog labels and got 100 percent training accuracy, it would be natural to say the model learned cats versus dogs. It might have. But training accuracy alone cannot prove that, because the same number can come from a model that learned a useful visual rule and from a model that found a way to fit the examples it saw.

The cleaner test is to destroy the semantic meaning of the labels while leaving the images intact. In this experiment, every training image receives a target label

$$y_i \in \{-1, +1\}.$$

The subscript i means “for example i .” So y_{17} is the random label assigned to image 17, independent of whether that image is a cat or a dog.

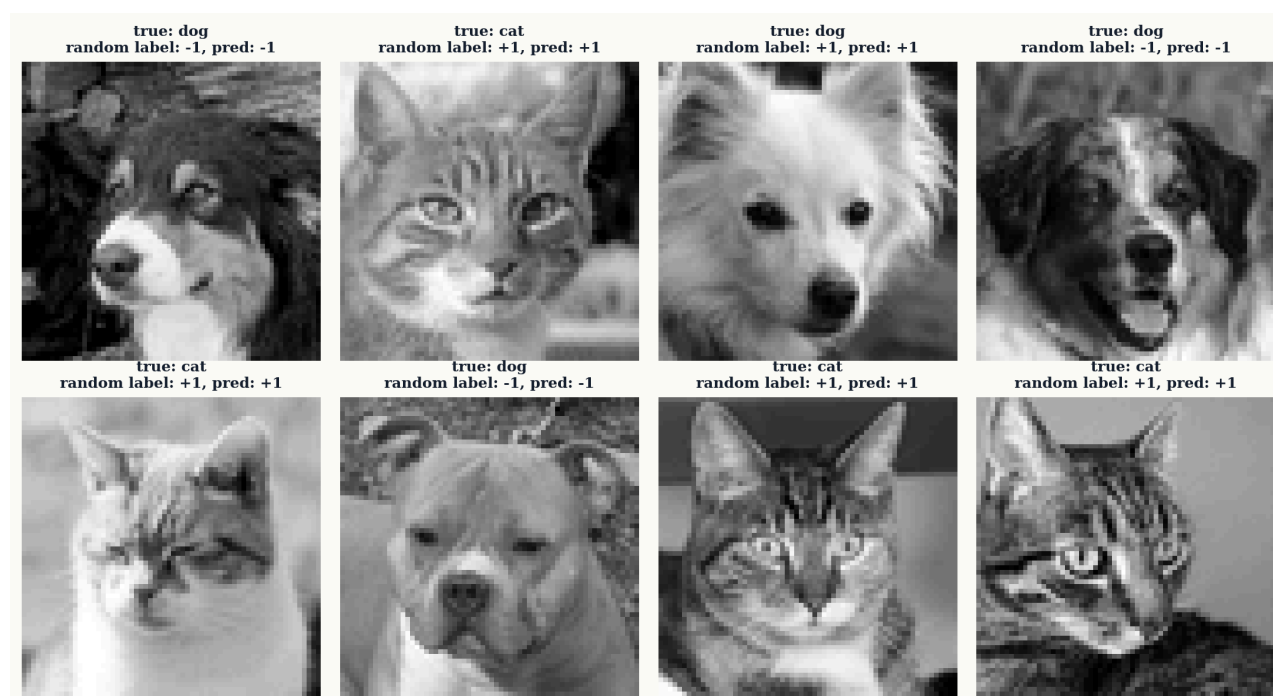


Figure 5: Real AFHQ cat and dog images with random training labels. The random label, not the true animal class, is what the perceptron is asked to fit.

If the perceptron fits those labels, the explanation cannot be that it learned the visual concept cat or dog. The label no longer encodes that concept. The model is fitting a random split of the training images, which turns the experiment into a capacity test rather than a test of animal recognition.

10 What is VC dimension?

VC dimension is a way to measure how flexible a class of models is. The formal theory is deeper than this chapter needs, but the central idea starts with a simple counting question. Take N points. If each

point can receive one of two labels, there are

$$2^N$$

possible labelings. For three points, that gives $2^3 = 8$ possible assignments; for ten points, $2^{10} = 1024$. The question behind VC dimension is whether a class of models can realize all of those assignments, not just one convenient assignment.

A model class is said to shatter a particular set of points if it can fit every possible binary labeling of those points. No matter how you assign $+1$ and -1 , some model in the class gets all the labels right.

The VC dimension is the size of the largest set of points the model class can shatter in the right geometric position. For linear separators in d input dimensions, the VC dimension is:

$$d + 1. \tag{3}$$

In this experiment:

$$d = 4096, \quad d + 1 = 4097.$$

Stated without the formal machinery, a linear classifier in 4096 dimensions can shatter 4097 points in general position. “General position” is a geometry phrase for points that have not fallen into a degenerate arrangement that removes independent directions. In the code, we do not simply assume the condition; we check it by measuring the rank of the augmented image matrix. Figure 6 shows why 4097 is the ceiling in this setup.

Put differently, VC dimension does not say the perceptron understands cats and dogs. It says the class of linear separators has enough capacity to realize any labeling of a sufficiently well-positioned set of up to 4097 image vectors. Random labels make that statement visible because they remove the semantic pattern that would otherwise distract from the geometry.

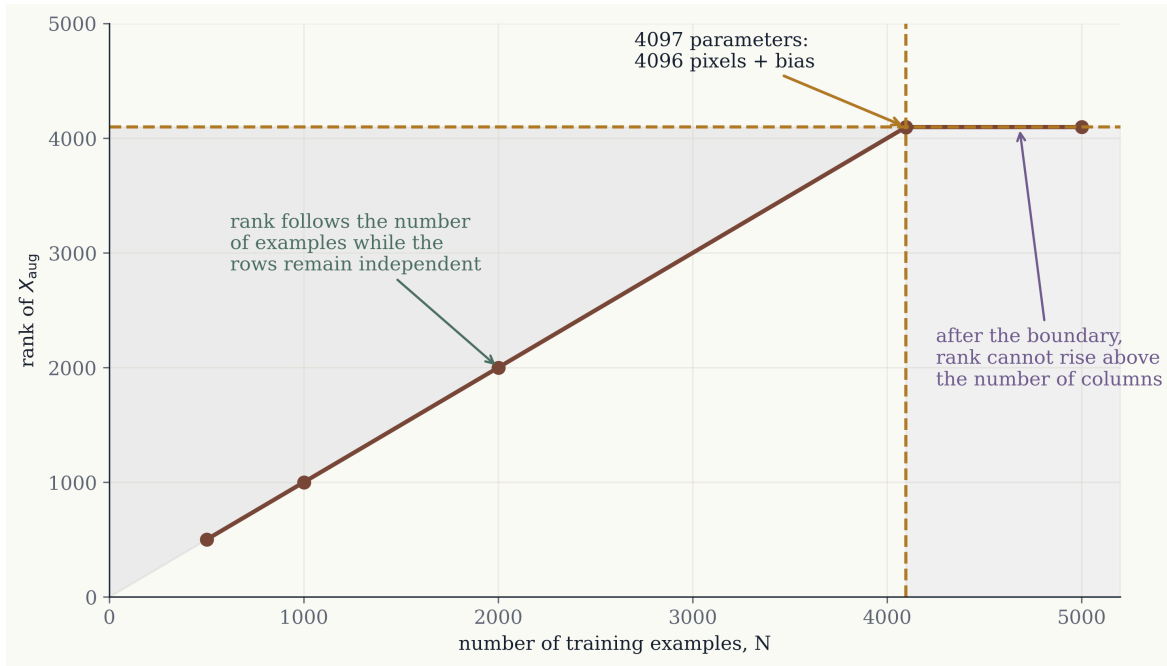


Figure 6: The augmented image matrix has one row per example and 4097 columns after the bias is added. In this sampled AFHQ run, the rank follows the number of examples through 4097 and then stops at the column limit, which is exactly the capacity boundary this chapter is testing.

11 Why the bias creates 4097 columns

The perceptron score is:

$$s = w^\top x + b.$$

The notation $w^\top x$ is shorthand for:

$$w_1x_1 + w_2x_2 + \cdots + w_{4096}x_{4096}.$$

The bias b is separate from the pixel weights in the usual formula, but it can be folded into the same vector by adding one extra input that is always equal to 1. For one image, define the augmented input as

$$x_{\text{aug}} = (x_1, x_2, \dots, x_{4096}, 1).$$

The word “augmented” only means that the extra constant value has been appended. Now define the augmented parameter vector as

$$\tilde{w} = (w_1, w_2, \dots, w_{4096}, b).$$

The tilde over $\tilde{w}$ is a reminder that this vector includes the bias. With this notation, the score becomes

$$s = x_{\text{aug}}^\top \tilde{w}.$$

This is the same score as before, only rewritten so the bias behaves like one more weight. For a whole dataset with N images, the augmented matrix X_{aug} has N rows and 4097 columns, with each row holding one image plus the final 1:

$$X_{\text{aug}} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,4096} & 1 \\ x_{2,1} & x_{2,2} & \cdots & x_{2,4096} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{N,1} & x_{N,2} & \cdots & x_{N,4096} & 1 \end{bmatrix}.$$

This is the source of the number 4097: the perceptron has 4096 pixel weights plus one bias, and the bias is what turns the image matrix into a 4097-column augmented matrix.

12 The linear-system construction

The phrase “linear system construction” can sound abstract, but it is only a compact way of writing down the requirement that every training score should match its label. For the first few images, that requirement would be

$$x_{1,\text{aug}}^\top \tilde{w} = y_1.$$

$$x_{2,\text{aug}}^\top \tilde{w} = y_2.$$

$$x_{3,\text{aug}}^\top \tilde{w} = y_3.$$

Instead of writing one equation per image for thousands of images, we stack the equations into one matrix equation:

$$X_{\text{aug}} \tilde{w} = y.$$

Figure 7 draws the same equation as a matrix so the dimensions are visible. X_{aug} is the data matrix, with one row per image and one column per parameter. In this experiment it has 4097 columns. $\tilde{w}$ is the vector of unknown parameters, meaning all pixel weights plus the bias. y is the vector of labels, with one random target for each image.

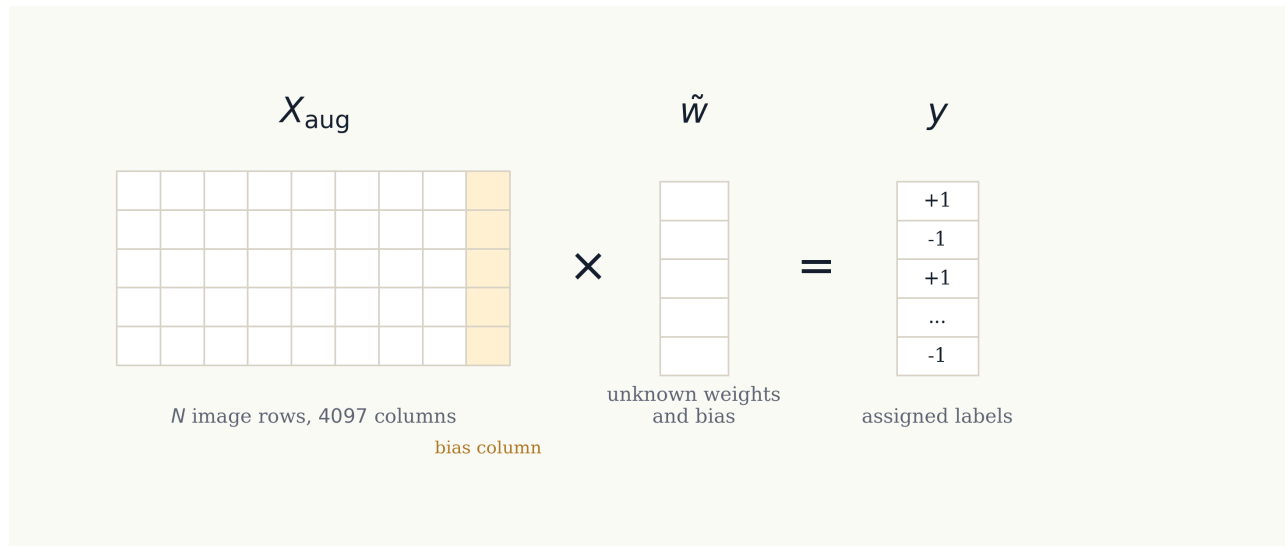


Figure 7: The exact separator proof writes all training-score equations at once. The augmented image matrix has one row per training image. The unknown vector contains the pixel weights and bias. The label vector contains the randomly assigned targets.

If the equation can be solved, then every score has the correct sign. A score of $+1$ predicts $+1$, and a score of -1 predicts -1 . Solving the linear system is therefore stronger than merely classifying the examples correctly, because it sets each score equal to the label itself.

13 Rank is the part that makes the proof work

The linear system $X_{\text{aug}} \tilde{w} = y$ is solvable for every possible label vector y when the rows of X_{aug} are linearly independent.

Rows are linearly independent when no row can be perfectly built from a weighted combination of the other rows. In this project, each row is an image. Full row rank means each training image contributes an independent direction in the augmented pixel space.

Rank is the number of independent rows or columns in a matrix. If X_{aug} has N rows and rank N , then all N rows are independent. We call this full row rank.

Why does that matter? Because full row rank means the map from parameter vectors $\tilde{w}$ to training scores can reach any target vector in $\mathbb{R}^N$. The weights have enough freedom to assign arbitrary scores to the training examples, not because they understand the images, but because the rows of the matrix give them enough independent directions to move in.

The proof chain is:

1. Each processed image is a 4096-dimensional vector.
2. Adding the bias creates a 4097-dimensional augmented vector.

3. Stacking N augmented images gives X_{aug} .
4. If $N \leq 4097$ and $\text{rank}(X_{\text{aug}}) = N$, the rows are independent.
5. Full row rank implies $X_{\text{aug}}\tilde{w} = y$ has a solution for any label vector y .
6. If the solution makes the scores equal $y_i \in \{-1, +1\}$, every score has the correct sign.
7. Therefore a linear separator exists for those labels.

This is the point where it is easy to mix up two claims. The rank construction proves that a separator exists. It does not show that the perceptron learning rule has already found it. That is why this project includes both the exact solve and the training run.

14 Why exact scores imply correct classification

Suppose the solve gives:

$$X_{\text{aug}}\tilde{w} = y.$$

For one image i , this says:

$$s_i = y_i.$$

The score s_i is the perceptron's raw score for image i . The label y_i is either $+1$ or -1 .

If $y_i = +1$, then $s_i = +1$, so the perceptron predicts $+1$. If $y_i = -1$, then $s_i = -1$, so the perceptron predicts -1 .

A compact way to write this is:

$$y_i s_i = y_i^2 = 1 > 0.$$

That expression says the label and the score have the same sign. When their product is positive, the point is correctly classified.

This is why the direct solve is so useful. It constructs scores that exactly match the labels, so the training error is zero.

15 The perceptron learning rule

The exact solve proves that a separator exists. The perceptron learning rule is the historical training algorithm that tries to find one through mistakes. It starts with weights at zero, loops through the training examples, and for each image computes:

$$s_i = w^\top x_i + b.$$

If the sign of s_i matches y_i , nothing changes. If the sign is wrong, the model updates its weights:

$$w \leftarrow w + \eta y_i x_i,$$

$$b \leftarrow b + \eta y_i.$$

The symbol η , pronounced eta, is the learning rate. It controls the size of the update.

The update is intuitive by cases. If the true label is $+1$ and the model predicts -1 , the score is too low. The update adds the image vector to the weights. Bright pixels in that image push future scores upward. If the true label is -1 and the model predicts $+1$, the score is too high. The update subtracts the image vector from the weights.

So the perceptron builds its weights out of signed copies of training images. Images labeled $+1$ tend to get added when they are misclassified. Images labeled -1 tend to get subtracted when they are misclassified.

This also explains why the final weight vector can be reshaped into a 64 by 64 image. There is one weight per pixel location.

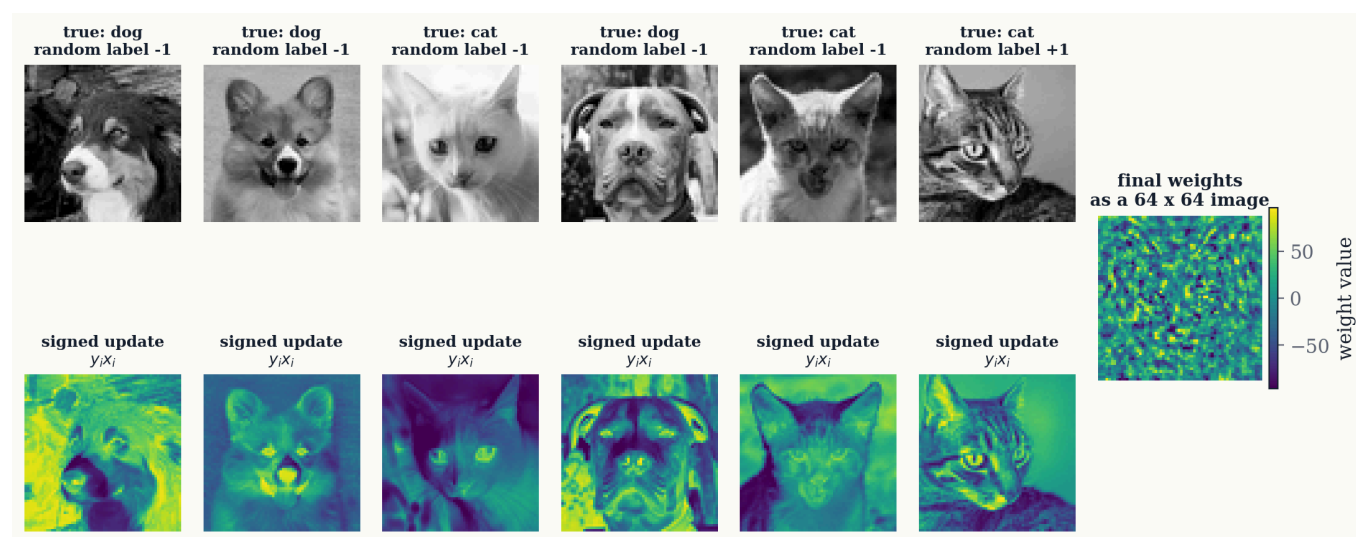


Figure 8: Perceptron weights are built from signed image-like updates. The final weight image is a memory trace of the random training split, not a clean cat or dog template.

The random-label setting is important here. The weight image in [Figure 8](#) should not be interpreted as a cat detector or dog detector. The labels are arbitrary. The weight image is a record of the corrections needed to separate this particular random split.

16 Why enough iterations does not give a simple epoch formula

The perceptron convergence theorem says that if the training data are linearly separable, the perceptron learning rule will eventually find a separator. The theorem connects the existence proof to the training process, but it is a guarantee of eventual success rather than a stopwatch.

In particular, the theorem does not say that 4097 examples should take exactly 4097 epochs, and it does not give a tight practical prediction for this dataset. The number of updates depends on the geometry of the examples.

The most important geometric quantity is the margin: the amount of breathing room between the separating hyperplane and the closest training points. A large margin means the classes are separated comfortably. A small margin means a separator exists, but some points sit very close to the boundary, so the algorithm may need many corrections before it finds a separating direction.

The classic perceptron mistake bound says that if every input has norm at most R , and there exists a unit-length separator with margin γ , then the perceptron makes at most:

$$\left(\frac{R}{\gamma}\right)^2$$

mistakes.

Here R is a bound on the size of the input vectors, and γ , pronounced gamma, is the margin. The important part is the ratio. If the margin is tiny, R/γ is large, and the mistake bound becomes large. Random labels usually create awkward geometry because they ask the model to thread a boundary through points with no underlying visual rule, which is why convergence can take far longer than the clean algebraic proof makes it sound.

The experiment therefore separates two responsibilities: the exact solve asks whether a separator exists at all, while perceptron training asks whether this particular update rule found one within the epoch budget we gave it.

1. VC dimension and rank tell us whether a separator exists.
2. The perceptron training run tells us how long this algorithm took to find one in this setup.

17 The experimental result

The first sweep trains for 50 epochs. The exact-separator column comes from the linear-system construction, while the perceptron column comes from finite perceptron training. The two columns

should be read together because they answer different questions: one asks what exists somewhere in parameter space, and the other asks what the mistake-driven algorithm found within a limited run.

Table 1: Fifty-epoch sweep on randomized AFHQ cat/dog labels.

N	rank of X_{aug}	exact-separator train error	perceptron train error after 50 epochs	perceptron converged?
500	500	0.0000	0.0000	yes
1000	1000	0.0000	0.0340	no
2000	2000	0.0000	0.1585	no
4096	4096	0.0000	0.2925	no
4097	4097	0.0000	0.2563	no
5000	4097	0.0262	0.2252	no

In Table 1, the rank column carries the existence proof. Through $N = 4097$, the rank equals N . The augmented image rows are independent, so the exact solve fits the random labels with zero training error all the way up to the 4097-example boundary.

The perceptron column tells the training story. At $N = 500$, the perceptron reaches zero training error within 50 epochs. At larger sample sizes, 50 epochs is not enough, which does not contradict the proof; it only says the training run stopped before the algorithm reached a separator. To test the “given enough iterations” part more directly, the experiment also runs longer perceptron training for the separable sample sizes:

Table 2: Long perceptron runs on randomized labels up to the VC-dimension boundary.

N	max epochs allowed	epochs to zero error	updates	converged?
500	50	48	2638	yes
1000	1000	80	9046	yes
2000	2000	214	44162	yes
4096	5000	945	365288	yes
4097	5000	1228	381320	yes

Table 2 is the direct demonstration. The exact solve proves that a separator exists through 4097 examples in this sampled dataset, and the long perceptron runs show the perceptron learning rule actually reaching zero training error at each of those sample sizes.

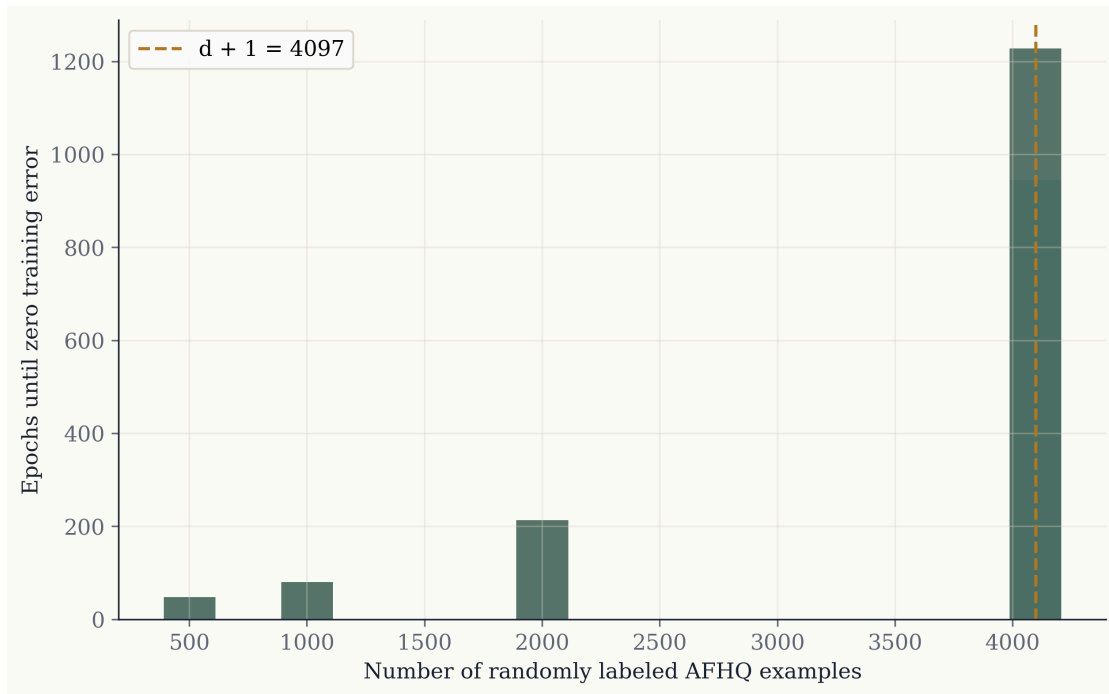


Figure 9: Long perceptron runs reach zero training error up through 4097 examples, though the number of epochs rises substantially.

The epoch counts are not universal constants. They depend on the random seed, preprocessing, example order, learning rate, and margin. The important result is not that $N = 4097$ took 1228 epochs specifically; it is that the perceptron converged on random labels at the VC-dimension boundary, exactly as the separability result says it can.

18 Why the 5000-example row changes

At $N = 5000$, the augmented matrix still has 4097 columns, so its rank cannot rise past 4097. In this run the rank is 4097 while the number of rows is 5000, which means the rows cannot all be independent. The exact linear solve therefore no longer has enough independent degrees of freedom to hit every random target exactly; its training error rises to 0.0262. Although the numerical error remains small, the qualitative change is the one that matters for the experiment: the fit is no longer perfect.

The VC-dimension boundary becomes visible here. The statement gives a shattering guarantee up to $d + 1$ examples under the right geometric condition. Once N is larger than $d + 1$, the guarantee has ended. Some particular labelings above the boundary may still be separable, because VC dimension is

about fitting every possible labeling of a set, not about ruling out every favorable case above the threshold. In this random-label run, 5000 examples are enough for the exact construction to lose perfect fit.

19 How to interpret the plots

The figures in this part of the chapter should be read as one sequence. The training-error plot compares the exact construction with finite perceptron training; the rank plot shows where the linear-algebra guarantee holds; the update plot shows how training cost changes as the sample size grows; and the $N = 500$ training journey makes the update process small enough to inspect closely.

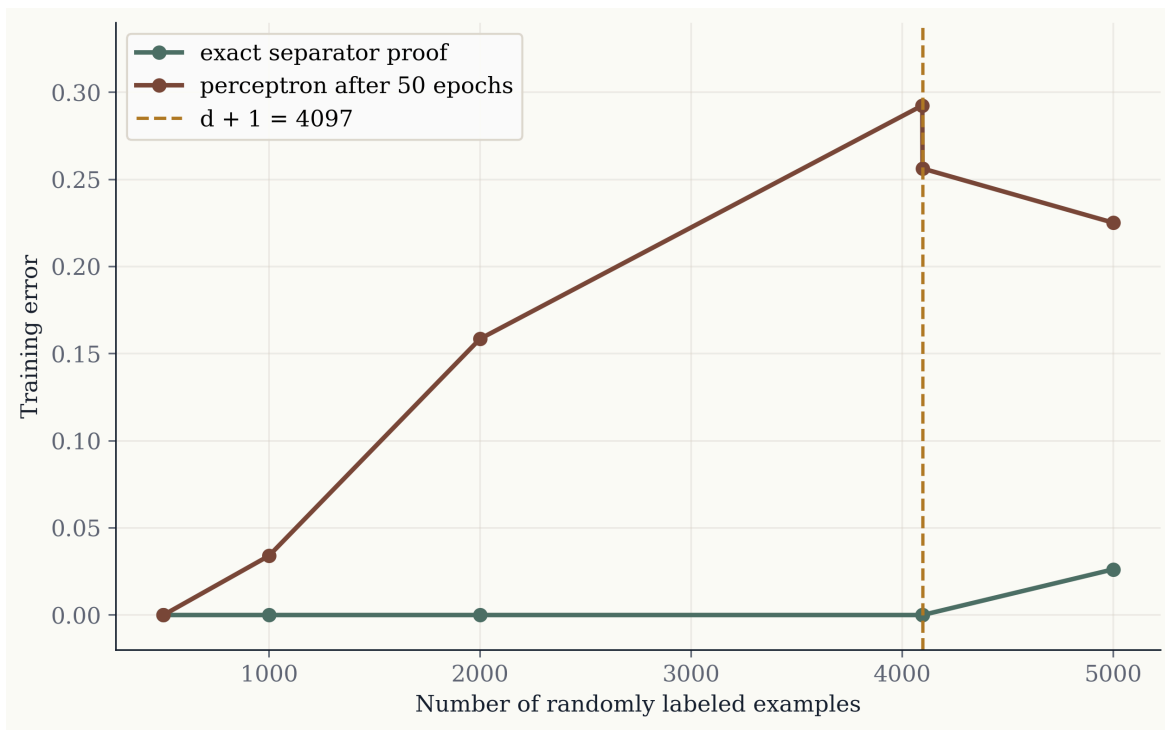


Figure 10: Training error across sample sizes. The exact separator has zero error through 4097 examples, while finite 50-epoch perceptron training does not converge for the larger sample sizes.

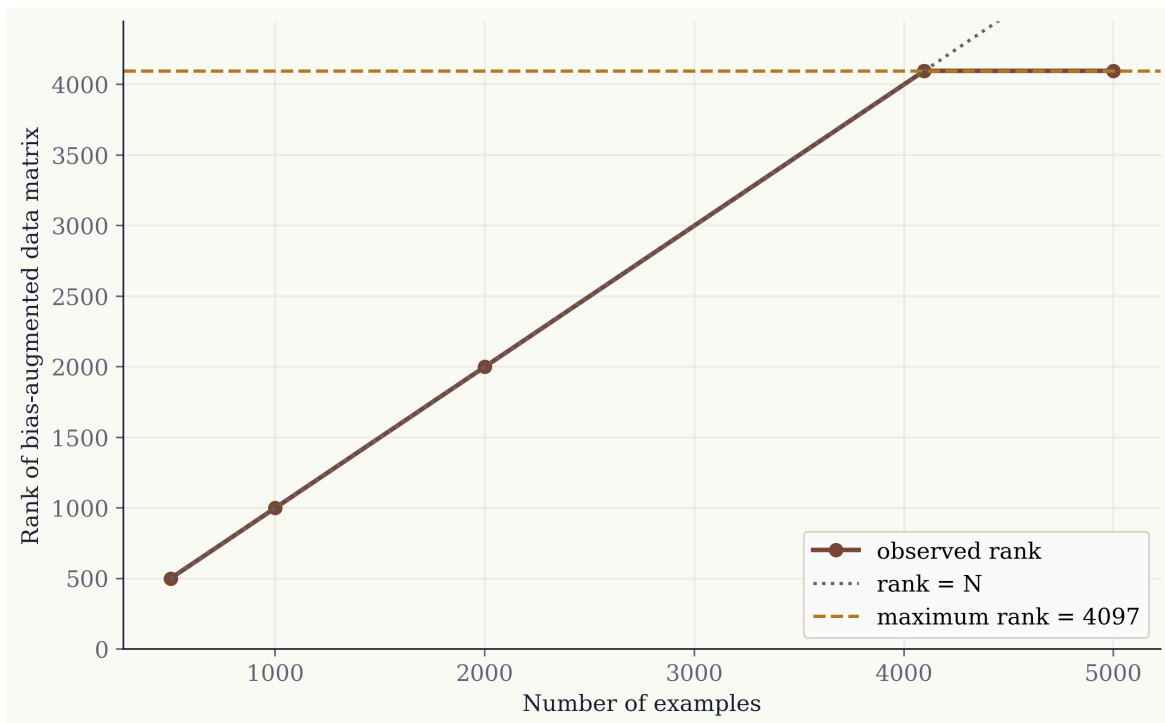


Figure 11: Rank versus sample size. The rank follows the number of examples until it reaches the 4097-column ceiling.

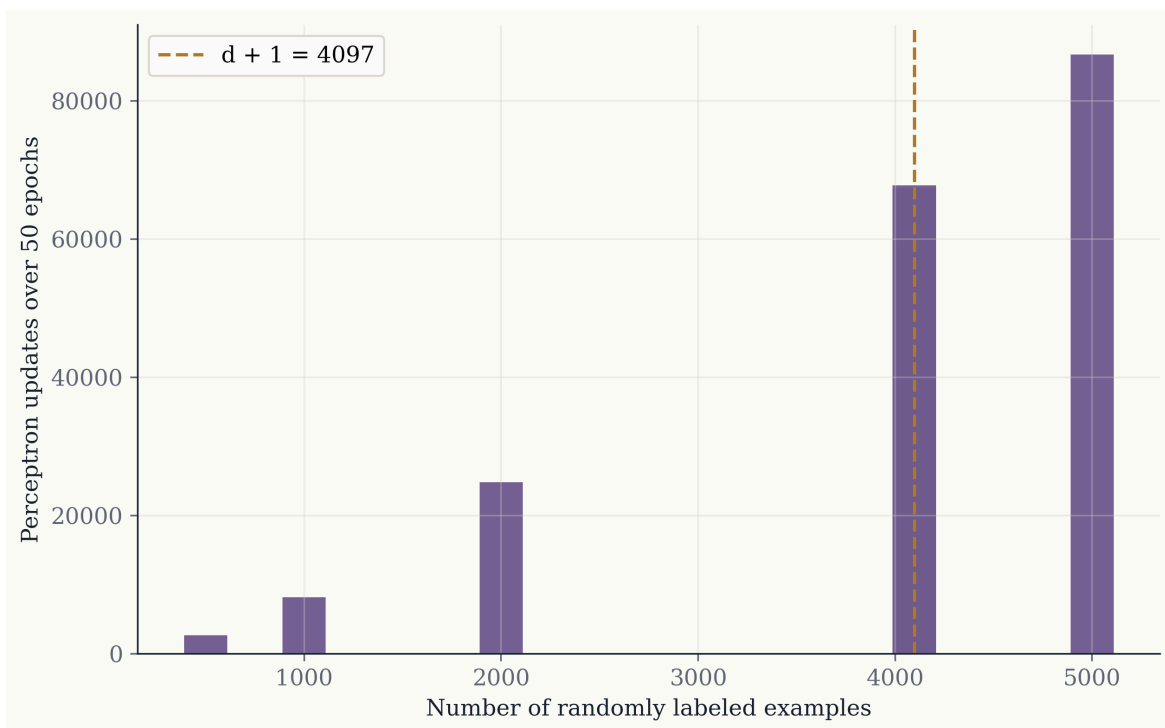


Figure 12: Perceptron updates after 50 epochs. Larger randomized training sets require many more corrections.

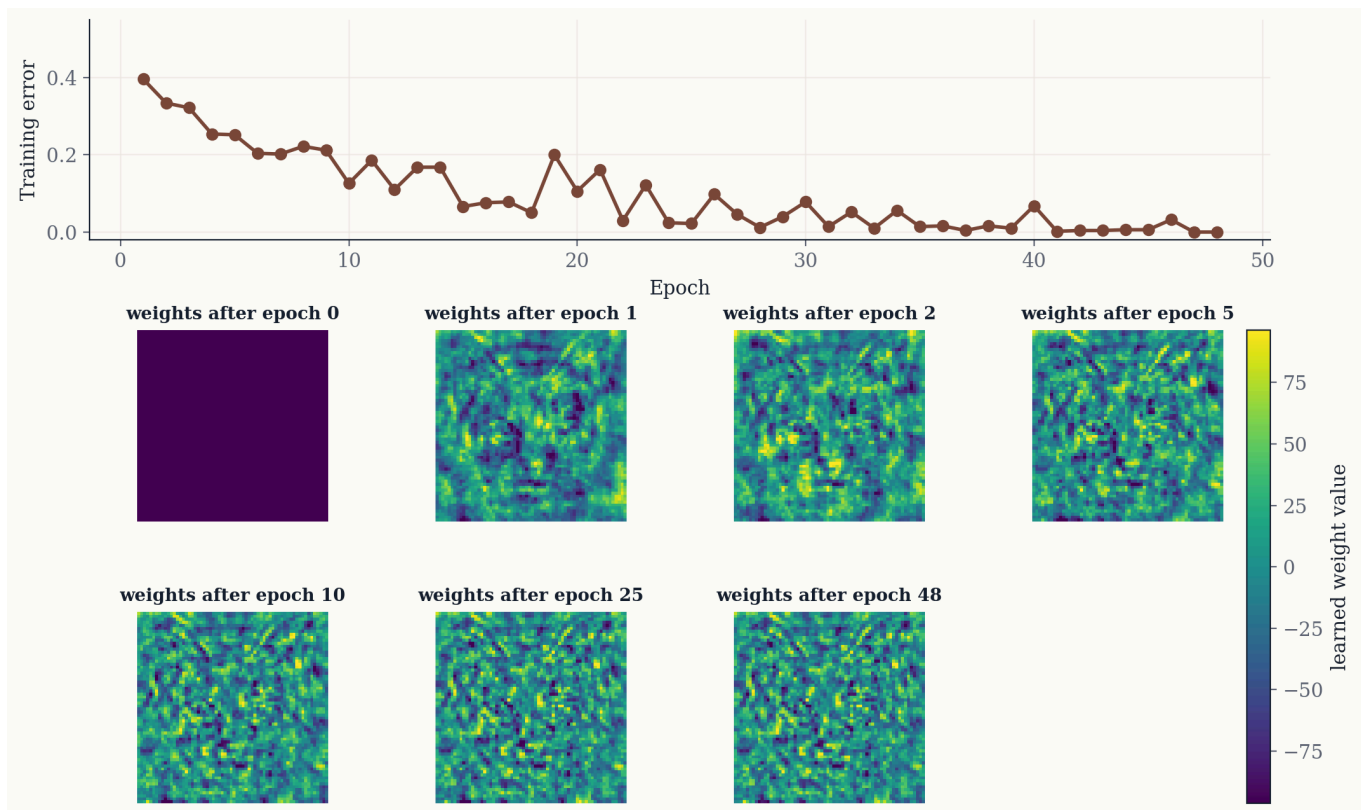


Figure 13: Training journey for 500 randomly labeled images, including snapshots of the evolving weight image.

The training journey makes the distinction concrete. The model starts wrong on many examples, each update moves the weights, and eventually the training error reaches zero. Because the labels were random, that success is not evidence of a learned animal concept. It is evidence that the model found a separating set of weights for this particular random split.

20 How the weights store information

It is tempting to say the perceptron stores the training set in its weights. A more careful version is that the learning rule leaves traces of the training examples inside the weight vector. The perceptron does not keep a database of images or filenames; instead, the final weight vector becomes a sum of signed training examples:

$$w = \sum_i \alpha_i y_i x_i.$$

Here x_i is training image i , y_i is its label, and α_i records how much that example contributed through mistakes and updates. If an example never caused an update, its contribution may be zero; if it caused multiple updates, its contribution may be larger. The final weight vector is therefore shaped by the examples that pushed on the model during training.

Because the weights are built from image vectors, the final w can be reshaped into a 64 by 64 image. In the random-label experiment, however, that image is not a cat template or a dog template. It is a signed accumulation of corrections: pixels from examples labeled $+1$ and pixels from examples labeled -1 have been added and subtracted until the training split becomes separable.

The best intuition is:

The weights are not learning catness or dogness under random labels. They are accumulating just enough signed pixel evidence to separate this particular random split.

21 What this does and does not prove

This experiment proves a specific capacity claim on a specific processed dataset. It does not show that all high-dimensional models only memorize, that neural networks cannot generalize, that real cat-versus-dog classification is impossible, or that VC dimension alone explains modern deep learning. Those would be much larger claims than this experiment can support.

The result is narrower, but it is durable: a high-dimensional linear classifier can fit random labels when the number of examples is small enough relative to the number of adjustable directions. The exact separator explains why the fit exists, and the long perceptron runs show that the original learning rule can actually find it when given enough iterations.

1. A high-dimensional linear model can fit arbitrary labels when the number of examples is small enough relative to the number of input dimensions and the data matrix has the right rank.
2. A perceptron can eventually find a separator when one exists, but the number of epochs depends on geometry.
3. Training accuracy alone cannot distinguish semantic learning from memorization.
4. Random labels are useful because they remove semantic meaning from the target.
5. Generalization requires more than fitting the training set.

That is the point that carries over to real machine learning. We care about performance on new examples, not only performance on the training set. If a model performs well only on the data it trained on, it may have memorized the examples without learning a rule that travels beyond them. Random labels are useful because they make that failure mode impossible to hide behind a familiar category name.

22 A hand exercise

Before trusting the 4096-dimensional result, solve a small version by hand.

Use the three points:

point	x_1	x_2	label
A	0	0	-1
B	1	0	+1
C	0	1	+1

Add the bias column:

$$X_{\text{aug}} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}.$$

Solve:

$$X_{\text{aug}} \tilde{w} = y.$$

This means:

$$\begin{aligned} b &= -1, \\ w_1 + b &= +1, \\ w_2 + b &= +1. \end{aligned}$$

Since $b = -1$, the other two equations give:

$$w_1 = 2, \quad w_2 = 2.$$

So:

$$\tilde{w} = (2, 2, -1).$$

The separator is:

$$2x_1 + 2x_2 - 1 = 0.$$

That is the same separator shown in [Figure 3](#). The large experiment is not using a different kind of reasoning. It is the same linear-system idea with 4097 columns instead of 3.

23 Practice questions

1. In your own words, explain why fitting random labels cannot be evidence that the model learned cats versus dogs.
2. A grayscale image is resized to 32 by 32. What is the input dimension? How many parameters does a perceptron have after adding a bias? What is the VC dimension of linear separators in that input space?
3. Suppose a dataset has $N = 100$ examples and $d = 20$ input dimensions. Can VC dimension guarantee that a linear classifier can shatter all 100 examples? Why or why not?
4. In the hand exercise, change the labels to $(+1, -1, +1)$. Solve the new linear system and write the separator.
5. Explain why the direct linear-system solve is not the same thing as perceptron training.
6. The long run reaches zero error for $N = 4097$ after 1228 epochs. Does VC dimension predict the number 1228? If not, what kind of information would help reason about training time?
7. If a model reaches 100 percent training accuracy and 50 percent test accuracy on balanced random labels, what does that tell you about memorization and generalization?
8. Look at the weight image. Why is it reasonable that the weights can be reshaped into an image? Why would it be misleading to call that image a cat detector in the random-label experiment?

24 Glossary

Bias. A parameter added to the weighted sum that shifts the decision boundary. In matrix form, it is handled by adding a constant column of 1s to the data.

Binary classification. A prediction task with two possible labels. This project uses -1 and $+1$.

Decision boundary. The set of input points where the model is exactly undecided. For a perceptron, this is where $w^\top x + b = 0$.

Epoch. One full pass through the training set.

Feature. One input variable. In this project, each pixel position is a feature.

Full row rank. A matrix with N rows has full row rank when its rank is N . This means the rows are linearly independent.

Hyperplane. The high-dimensional version of a line or plane. A perceptron's decision boundary is a hyperplane.

Linearly separable. A dataset is linearly separable if some hyperplane can place all positive examples on one side and all negative examples on the other.

Margin. The amount of breathing room between a separator and the closest training examples. Larger margins usually make perceptron learning easier.

Perceptron. A linear classifier that predicts from the sign of a weighted sum.

Random labels. Labels assigned independently of the real class. They are useful for testing memorization because they remove semantic meaning.

Rank. The number of independent directions represented by the rows or columns of a matrix.

Shattering. A model class shatters a set of points if it can fit every possible binary labeling of those points.

Training error. The fraction of training examples classified incorrectly.

VC dimension. A measure of model capacity. For linear separators in d dimensions, the VC dimension is $d + 1$.

25 Source note

This project was inspired by the perceptron and VC-dimension discussion in *The Welch Labs Illustrated Guide to AI*. The repository turns that idea into a self-contained experiment and learning module using AFHQ animal-face images.